

STANDARD OPERATING PROCEDURE (SOP)

Engineering Coordination Center for Frappe Framework & ERPNext (Frappe ECC)

Document ID	SOP-ENG-FRAPPE-ECC-001
Version & Release	Version 1.0.0 (Production Release)
Effective Date	2026-09-18
Applicability	Frappe v14 / v15 / v16, ERPNext, Antigravity, Claude Code, Cursor

1. Purpose & Scope

This Standard Operating Procedure defines the mandatory engineering practices, execution phases, and automated quality controls for building, testing, and securing Frappe Framework applications using Frappe ECC. Frappe ECC turns your AI coding assistant (Antigravity, Claude Code, Cursor, Codex) into a fully coordinated engineering system equipped with 12 specialized agents, 14 production workflow skills, 13 slash commands, and Frappe Shield.

2. System Architecture & Component Inventory

Component Layer	Count	Core Responsibilities
Specialist AI Agents	12	Planner, Architect, Controller Builder, Desk UI Builder, TDD Guide, Code Reviewer, Security Reviewer, Bench DevOps, Migration Patcher, Report Builder, API Integrator, Doc Updater
Workflow Skills	14	DocType modeling, QueryBuilder/ORM, hooks.py, Desk Client Scripts, REST API, Permissions, TDD with FrappeTestCase, RQ background jobs, Script Reports, Patches, Bench CLI, Frappe UI, Web Portal, Security Audit
Slash Commands	13	Quick entry points: /frappe:plan, /frappe:doctype, /frappe:controller, /frappe:client-script, /frappe:hook, /frappe:api, /frappe:test, /frappe:review, /frappe:security, /frappe:patch, /frappe:report,

		/frappe:bench, /frappe:help
Coding Rules	5	Always-loaded standards: Core architecture, Python backend, Desk JS, Security/Permissions, and MariaDB/PostgreSQL database optimization
Frappe Shield	1 CLI	AST-based static analyzer detecting SQL injection in frappe.db.sql, db.commit() transaction violations, unvalidated guest endpoints, and N+1 query bottlenecks

3. Installation & Verification Procedure

Follow these exact commands to install Frappe ECC into your code editor:

[RECOMMENDED SETUP] Antigravity installation automatically registers skills into your global config directory so the AI assistant recognizes all Frappe paradigms across every project.

Command for Linux / macOS / WSL:

```
./install.sh --profile minimal --target antigravity
```

Command for Windows PowerShell:

```
.\install.ps1 -Profile minimal -Target antigravity
```

Verification Check:

```
python bin/frappe_ecc_install.py doctor
```

4. The 10-Phase Feature Development SOP

Every engineer and AI assistant MUST adhere to the following 10-phase sequence:

Phase 1: Feature Planning & Architecture Blueprinting

Never write code without a blueprint. Invoke the planner agent using `/frappe:plan` or ask your assistant. The planner determines DocType taxonomy (Standard vs Child Table vs Single vs Submittable), fieldtypes, autoname rules (format:AST-YYYY-#####), module boundaries, and execution order.

Phase 2: Schema Modeling & DocType Scaffolding

Generate schema JSON, controller, client script, and test files using `/frappe:doctype`. Ensure search indexes (`'search_index': 1`) are applied to frequent lookup fields. Embed child tables via `'fieldtype': 'Table'` and `'options': '<Child DocType>'`.

Phase 3: Test-Driven Development (TDD) — Write Tests First

Before implementing business logic, write failing unit tests in `'test_<doctype>.py'` with `FrappeTestCase`. Assert autoname prefix, mandatory fields, validation failures, and transaction rollbacks. Run tests using `'bench run-tests --doctype <DocType>'` to confirm RED failing state.

Phase 4: Backend Controller & Lifecycle Implementation

Implement controller logic using `/frappe:controller`. Follow Frappe's exact lifecycle: `before_insert`, `validate`, `on_submit`, and `on_cancel`. CRITICAL: NEVER call `frappe.db.commit()` inside controller events. Use `frappe.qb` (QueryBuilder) for complex queries. Wrap user strings with `_('...')`.

Phase 5: Desk UI & Reactive Client Scripting

Construct responsive Desk forms using `/frappe:client-script`. Use `'frappe.ui.form.on'` for events: `onload`, `refresh`, and field triggers. Use `frm.add_custom_button`, `frm.set_df_property`, and native dialogs. CRITICAL: Never manipulate the DOM directly using jQuery or raw selectors.

Phase 6: Hook Registration & Event Orchestration

Connect application events, cron jobs, and class overrides into `hooks.py` using `/frappe:hook`. Configure `doc_events` for cross-datatype triggers, `scheduler_events` for cron routines, and fixtures for persistent custom fields and roles.

Phase 7: Whitelisted APIs & Webhook Integrations

Create secure REST endpoints using `/frappe:api`. Annotate endpoints with `@frappe.whitelist()`. Enforce `frappe.only_for()` role authorization. Add `@frappe.rate_limit` on public endpoints. Sanitize inputs with `frappe.utils.escape_html()`.

Phase 8: Database Schema Migrations & Patches

Create idempotent migration patches in `'patches/vX_Y/<patch>.py'` using `/frappe:patch`. Always call `frappe.reload_doc()` before database writes. Verify with `'bench migrate'`.

Phase 9: Fresh-Context Code Review

Execute an independent, fresh-context code review using `/frappe:review`. Detects stray `db.commit()` calls, N+1 query bottlenecks, unindexed filters, and untranslated strings.

Phase 10: Security Audit with Frappe Shield

Run automated AST security scanning using `python bin/frappe-shield.py <path>` or `/frappe:security`. Flags SQL injection in `frappe.db.sql`, transaction violations, and direct docstatus mutations. All CRITICAL and HIGH issues must be resolved before deployment.

5. Frappe Shield Security Rules & Remediation

Vulnerability Pattern	Severity	Remediation Standard
SQL Injection: f-string or % in <code>frappe.db.sql()</code>	CRITICAL	Replace string interpolation with <code>values={'key': val}</code> or migrate to <code>frappe.qb</code> .
Manual DB Commit Inside Controller Hook	CRITICAL	Remove <code>frappe.db.commit()</code> . Frappe manages transaction commits automatically.
Unrestricted Guest Whitelist API	HIGH	Add <code>@frappe.rate_limit(limit=10, seconds=60)</code> and validate/sanitize all inputs.
Direct docstatus = 1 Assignment	HIGH	Use <code>doc.submit()</code> or <code>doc.cancel()</code> instead of directly overwriting docstatus.
N+1 Query Loop (<code>frappe.get_doc</code> in loop)	MEDIUM	Batch fetch using <code>frappe.get_all(filters={'name': ['in', ids]})</code> or <code>frappe.db.get_values()</code> .

6. DevOps & Bench Troubleshooting Runbook

- **Redis Connection Failure (`redis.exceptions.ConnectionError`):** Ensure Redis is running: `'sudo systemctl restart redis-server'`. Verify socket path in `config/redis_cache.conf`.
- **Duplicate Entry Error on Schema Migration:** Inspect table using `'bench mariadb'`. Remove or update duplicate rows prior to applying unique index constraint.
- **UI Changes Not Reflecting in Desk Form:** Clear Redis cache and rebuild assets: `'bench --site <site> clear-cache && bench build --app <app>'`. Perform hard browser refresh (Ctrl+F5).

- **Background Workers Stalled / Queues Backlogged:** Inspect queue health with 'bench doctor'. Restart RQ workers: 'bench worker --queue default,short,long'.

7. Quick Reference Command Cheat Sheet

Slash Command	Specialist Agent	Standard Operation
/frappe:plan '<feature>'	frappe-planner	Architect feature schema blueprint and relationships
/frappe:doctype '<Name>'	frappe-planner	Scaffold complete .json, .py, .js, and test files
/frappe:controller '<Name>'	frappe-backend-builder	Implement controller lifecycle hooks and validations
/frappe:client-script '<Name>'	frappe-desk-builder	Construct reactive Desk form scripts and custom dialogs
/frappe:hook [event cron]	frappe-architect	Wire doc_events, scheduler cron, or class overrides
/frappe:api '<name>'	frappe-api-integrator	Generate secure whitelisted REST API endpoint
/frappe:test '<Name>'	frappe-tdd-guide	Scaffold unit tests with FrappeTestCase
/frappe:review [path]	frappe-code-reviewer	Review diffs in fresh context for Frappe anti-patterns
/frappe:security [path]	frappe-security-reviewer	Execute AST vulnerability audit (SQLi, IDOR, XSS)
/frappe:patch '<desc>'	frappe-migration-patcher	Draft idempotent database migration patch
/frappe:report '<name>'	frappe-report-builder	Scaffold Script Report (Python backend + JS frontend)
/frappe:bench [task]	frappe-bench-devops	Diagnose bench errors and site operations
/frappe:help	All Agents	Display command cheat sheet and quick guidance

APPROVED & ENFORCED BY: Antigravity Systems Engineering Group

DISTRIBUTION: Open Source / All Engineering Teams